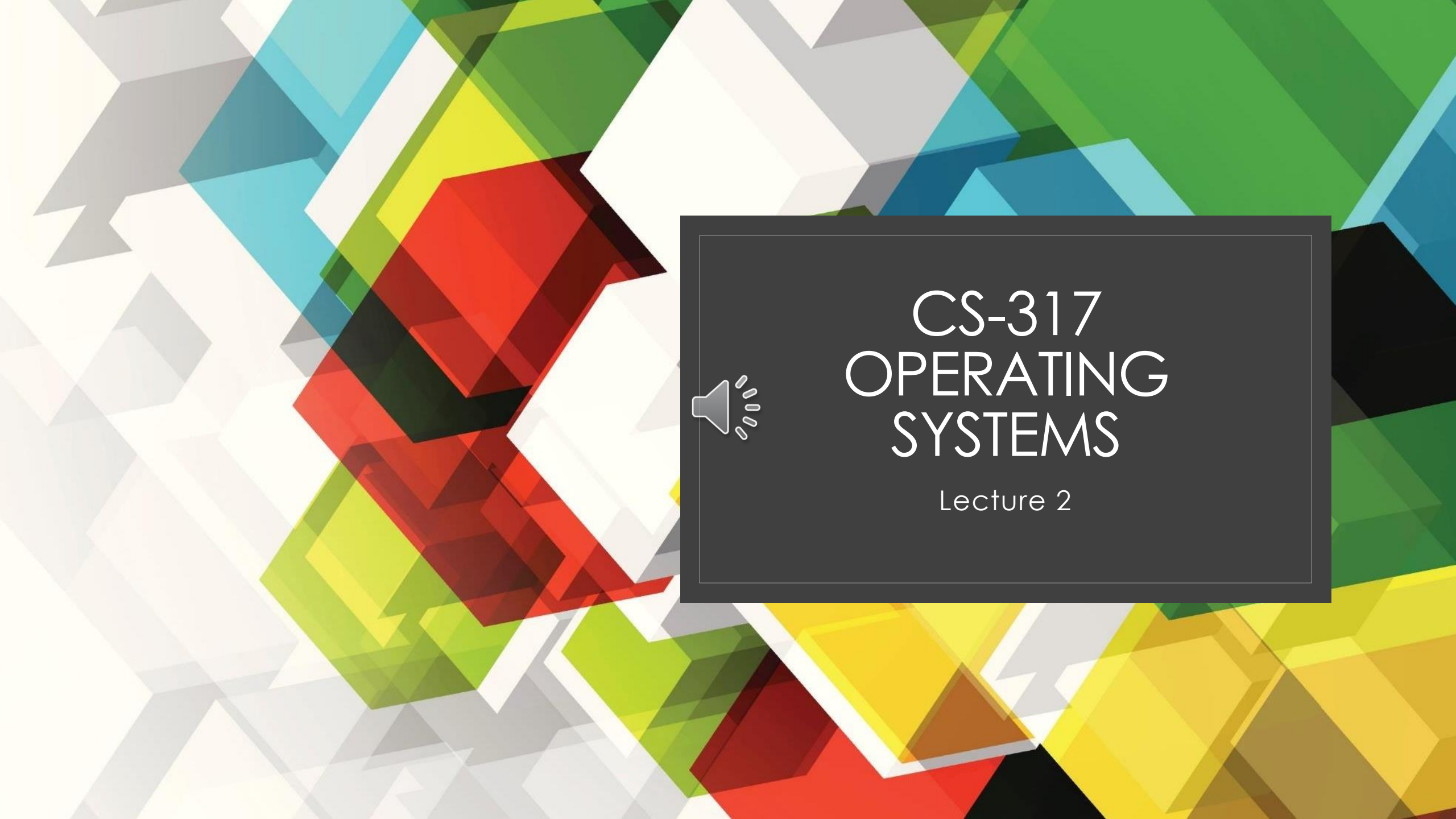




CS-317 OPERATING SYSTEMS

Lecture 2



PROCESSES

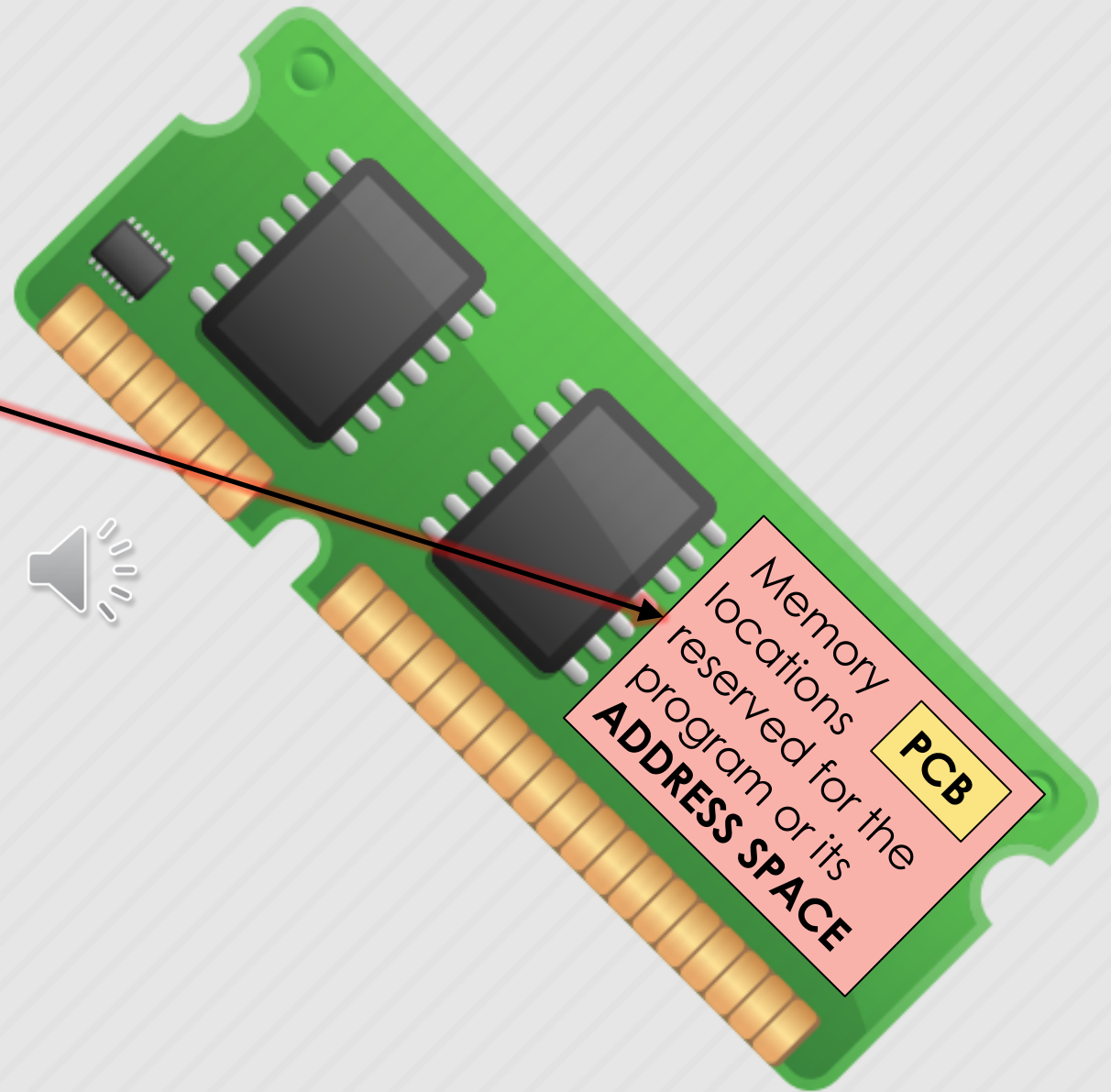
Book 1 Chapter 3

What is a process?

- A process is a program in execution.
- The program is a set of instructions stored on secondary storage.
- It is important to remember that execution of any entity can only take place after it has been loaded in primary storage.
- On the secondary storage, the program is a static entity, because it cannot execute.
- When the user asks for a program to be executed, the OS
 - Assigns an ID to it and creates some data structures for it,
 - Allocates a set of memory locations for the program in main memory, and
 - Loads the program code from the secondary storage to the primary storage.
- Memory locations allocated to a program are called its **address space**. Once the program has been allocated an address space in the primary storage, it becomes a dynamic entity, as it can now execute.
- A program becomes a **process** when the OS decides to execute it.



Data structures for the program or the **PROCESS CONTROL BLOCK**



Reasons for process creation



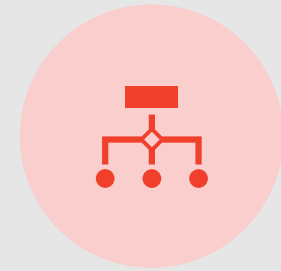
NEW BATCH JOB



INTERACTIVE LOG-
ON



CREATED BY OS TO
PROVIDE A SERVICE



SPAWNED BY
EXISTING PROCESS

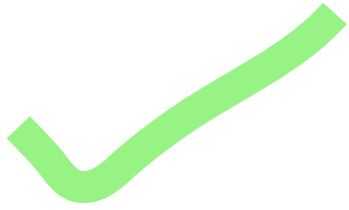


Table 3.1 Reasons for Process Creation

New batch job	The OS is provided with a batch job control stream, usually on tape or disk. When the OS is prepared to take on new work, it will read the next sequence of job control commands.
Interactive log-on	A user at a terminal logs on to the system.
Created by OS to provide a service	The OS can create a process to perform a function on behalf of a user program, without the user having to wait (e.g., a process to control printing).
Spawned by existing process	For purposes of modularity or to exploit parallelism, a user program can dictate the creation of a number of processes.

Reasons for
process
creation

Reasons for process termination

Normal
Completion

Time limit
exceeded

Time
overrun

Memory
unavailable

Bounds
violation

Protection
error

I/O failure

Arithmetic
error

Invalid
instruction

Privileged
instruction

Data misuse

Operator or
OS
intervention

Parent
termination

Parent
request



Table 3.2 Reasons for Process Termination

Normal completion	The process executes an OS service call to indicate that it has completed running.
Time limit exceeded	The process has run longer than the specified total time limit. There are a number of possibilities for the type of time that is measured. These include total elapsed time ("wall clock time"), amount of time spent executing, and, in the case of an interactive process, the amount of time since the user last provided any input.
Memory unavailable	The process requires more memory than the system can provide.
Bounds violation	The process tries to access a memory location that it is not allowed to access.
Protection error	The process attempts to use a resource such as a file that it is not allowed to use, or it tries to use it in an improper fashion, such as writing to a read-only file.
Arithmetic error	The process tries a prohibited computation, such as division by zero, or tries to store numbers larger than the hardware can accommodate.
Time overrun	The process has waited longer than a specified maximum for a certain event to occur.
I/O failure	An error occurs during input or output, such as inability to find a file, failure to read or write after a specified maximum number of tries (when, for example, a defective area is encountered on a tape), or invalid operation (such as reading from the line printer).
Invalid instruction	The process attempts to execute a nonexistent instruction (often a result of branching into a data area and attempting to execute the data).
Privileged instruction	The process attempts to use an instruction reserved for the operating system.
Data misuse	A piece of data is of the wrong type or is not initialized.
Operator or OS intervention	For some reason, the operator or the operating system has terminated the process (e.g., if a deadlock exists).
Parent termination	When a parent terminates, the operating system may automatically terminate all of the offspring of that parent.
Parent request	A parent process typically has the authority to terminate any of its offspring.

Reasons for
process
termination

Task# 1: What is the difference between time limit exceeded and time overrun?



Task# 2: What is the difference between invalid instruction and data misuse?

Pause the video!

Write your answer in a word file. Save it as <my_roll_no>_lecture_2. If your roll no is 500, the file will be named 500_lecture_2. Do not add <> signs to the filename.



PROCESS STATES

Different types of a process based on how much it has been executed and where it is placed

Models to define process states



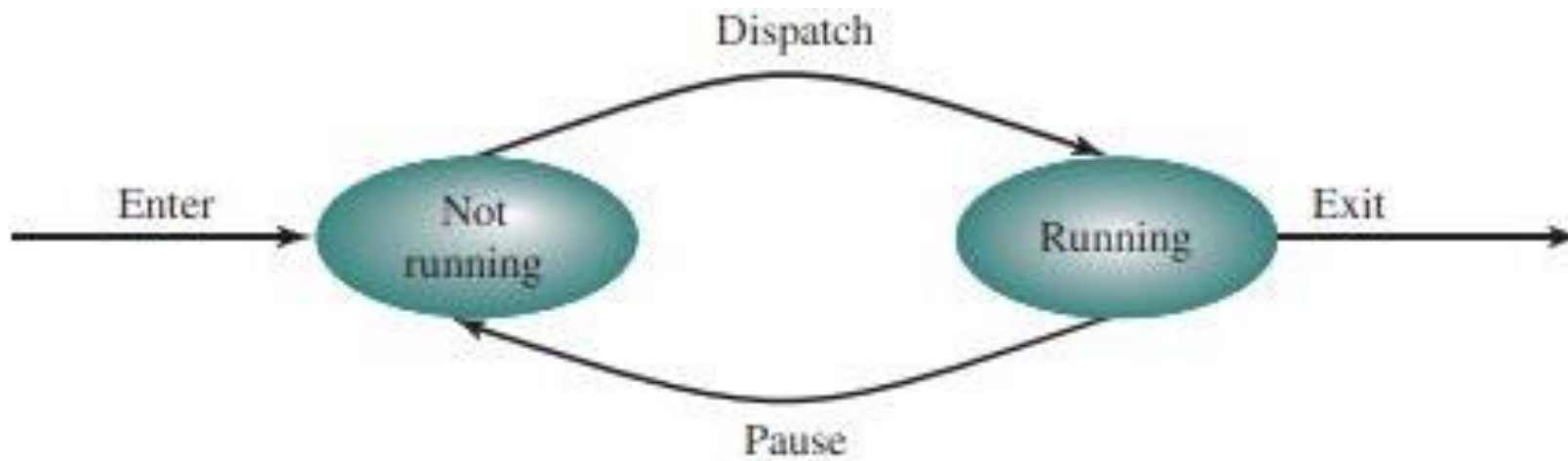
2-state process
model



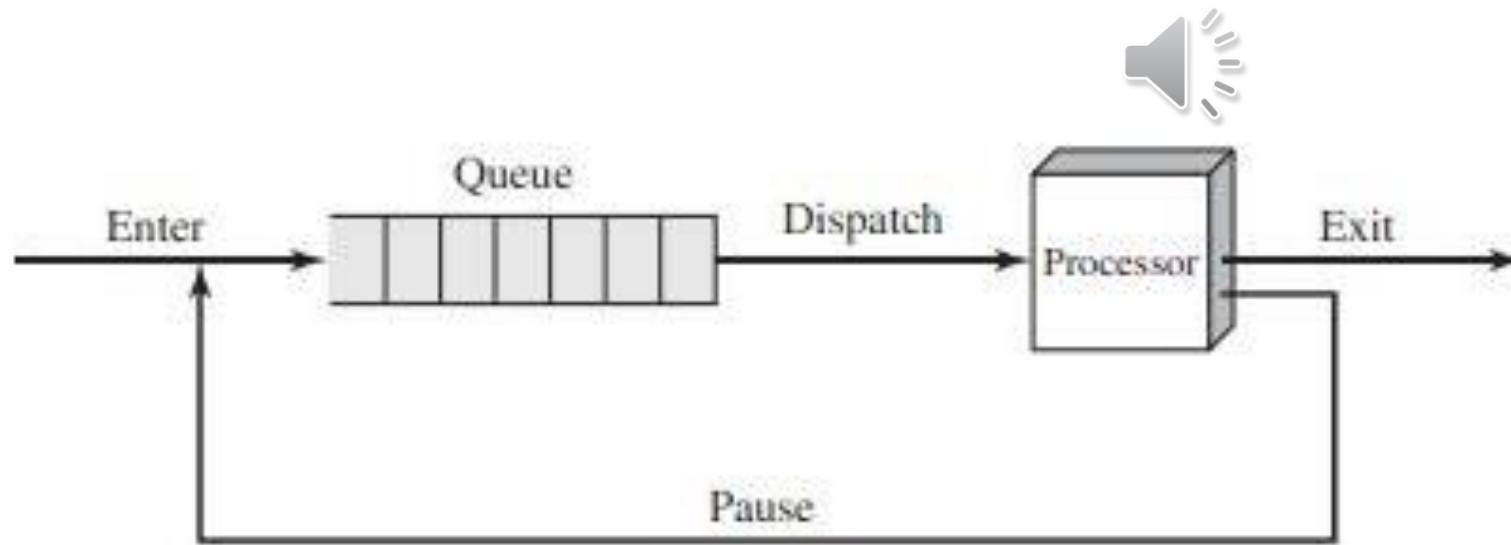
5-state process
model



7-state process
model



(a) State transition diagram



(b) Queueing diagram

2-state process model

- **Not running:** The process has not been allocated the processor.
- **Running:** The process has been allocated the processor by the **dispatcher**. In this state, the process is executing.

Figure 3.5 Two-State Process Model

Points to note

- Each slot in the queue contains a **pointer to the PCB of a process**.
- When control is transferred to the process, **its context is restored**, that is, the OS copies the process' variables from its address space into the processor's registers; also, the OS reads the process' PCB and sets the program counter to the address of the instruction to be executed .
- When the time slot expires, **the context of the process is saved** by the OS, that is, its variables are copied back from the processor's registers into its address space, and the OS records the address of the last instruction executed in its PCB.

General structure of a job



- Every user job is composed of CPU bursts and I/O bursts. In a CPU burst, the job executes on the processor. In an I/O burst, an I/O operation that the job requested takes place.
- The red chunks are I/O bursts. The yellow chunks are CPU bursts.
- Every job starts and ends with a CPU burst.
- Every I/O burst is sandwiched between two CPU bursts.
- Jobs are either CPU intensive or I/O intensive. CPU intensive jobs have long CPU bursts and short I/O bursts. I/O intensive jobs have short CPU bursts and long I/O bursts.
- When a job requests I/O, the job waits for the operation to finish. During this time, the processor is idle.

Problem with 2-state process model

- Consider that a process is at the head of the queue. The dispatcher selects it and grants the processor to it. The job executes for a while and then asks for an I/O operation. Then it relinquishes the processor. Control is transferred back to the dispatcher, which is a part of the OS, and it places this job at the end of the queue.
- After some time, you will have two kinds of processes in the queue, processes which are ready to execute, and processes which are waiting for any I/O event to occur. A process will not resume execution as long as the I/O event it asked for is not complete.
- We call a process that is waiting for an I/O event a blocked process. The OS records in its PCB the ID of the event it is waiting for. When the relevant I/O device completes its task, it interrupts the processor. This interrupt is inspected by the OS. It determines which processes are waiting for this event, and if necessary, it places data from the I/O device in their address spaces, and it records in their PCBs that the event is complete.
- Now the dispatcher cannot simply pick a process from the head of the queue, because the process at the head of the queue may be a blocked process. The dispatcher must scan all PCBs starting from the head of the queue until it finds a process that is ready to execute. This increases delay at the dispatcher. During this time, the processor sits idle.

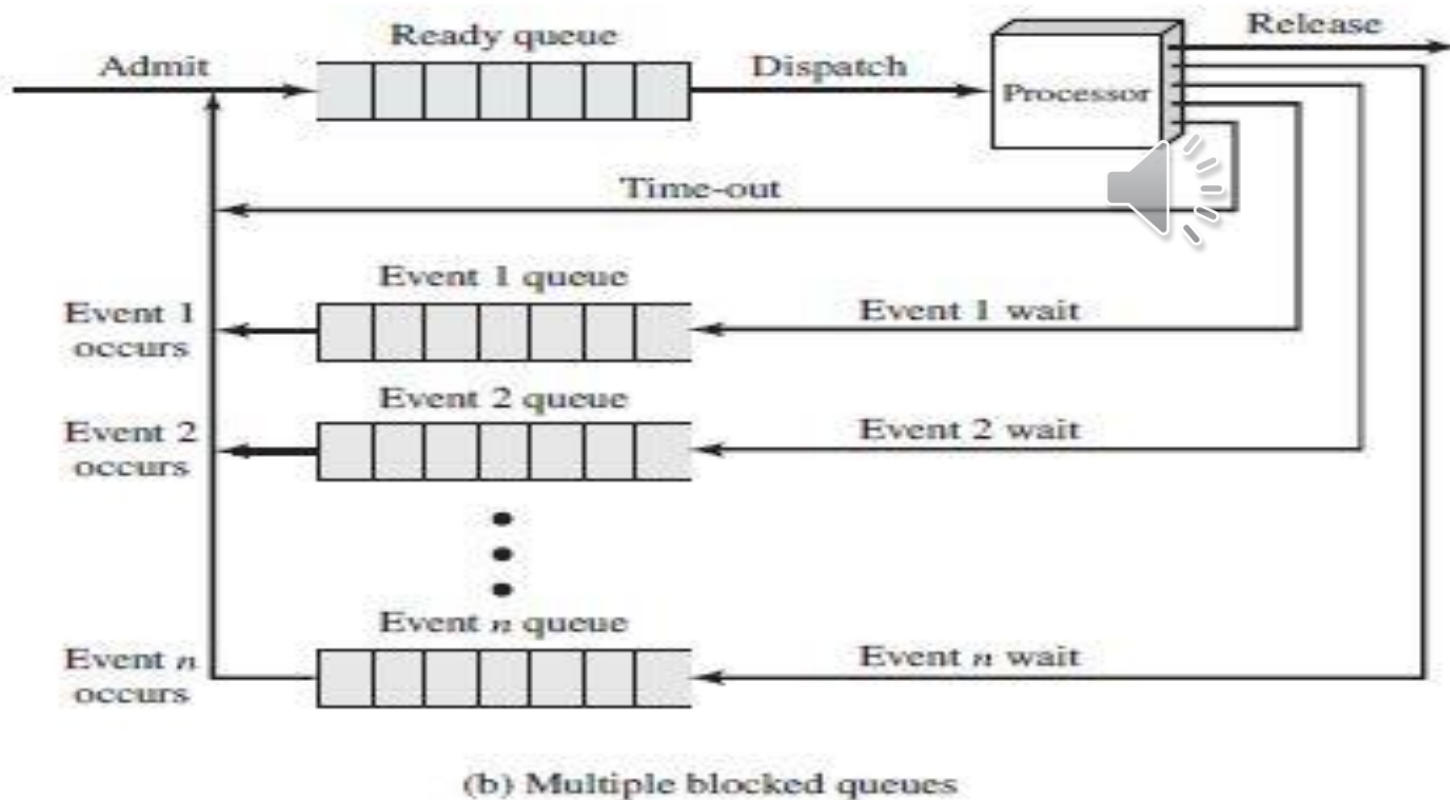
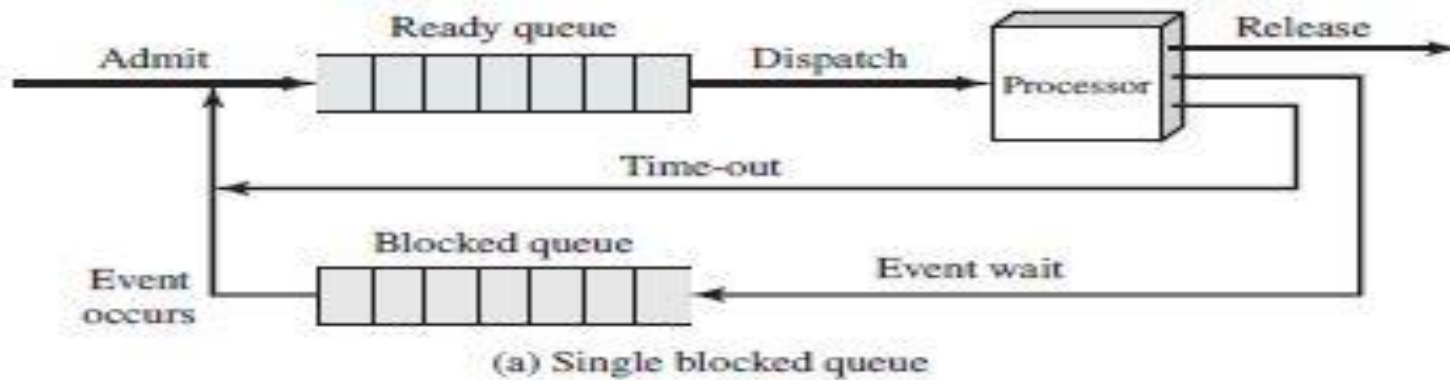


Figure 3.8 Queuing Model for Figure 3.6

Queues in the 5-state process model

- We use two queues in this model.
- The ready queue contains processes that are ready to execute.
- The blocked queue contains processes that are blocked for an I/O event.

States in the 5-state process model



NEW



READY



RUNNING



EXIT



BLOCKED

5-state process model

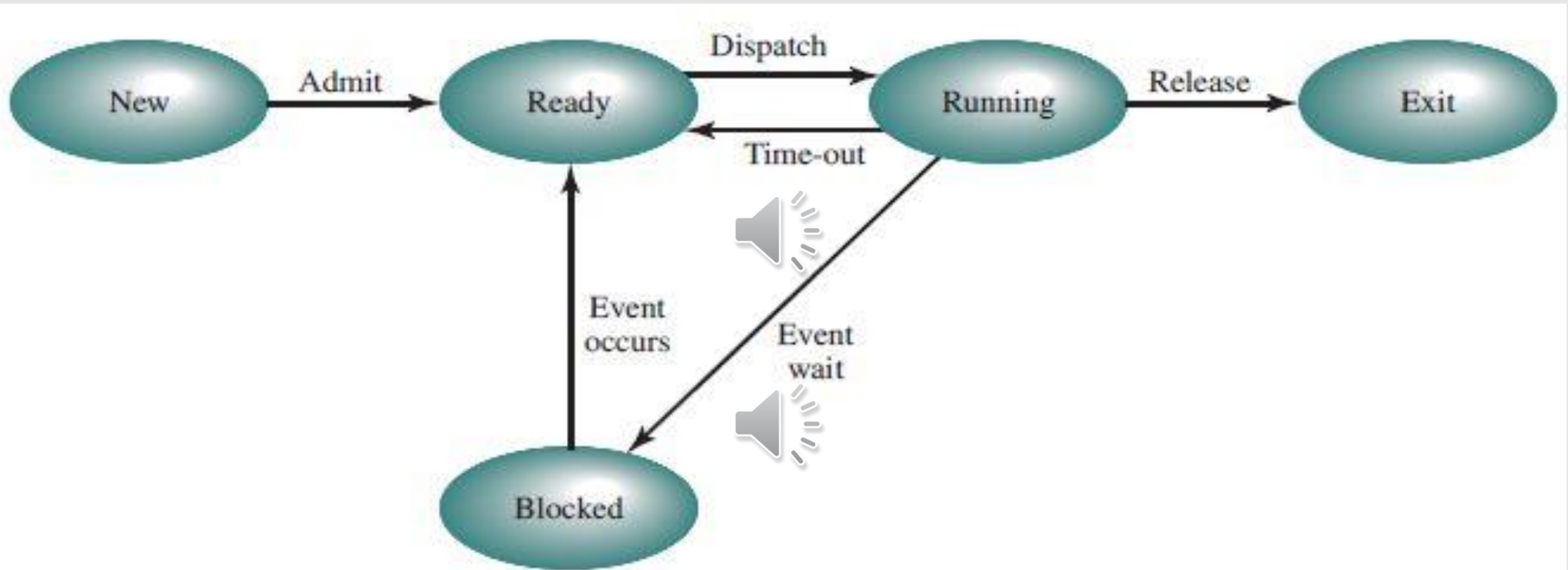
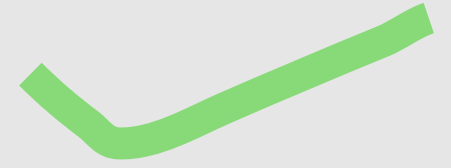


Figure 3.6 Five-State Process Model

Task# 3: Consider a system with priorities. A high priority process named A is currently blocked. A low priority process named B is running. An interrupt signals  completion of the event for which A is waiting. What should be the transitions in the system, so that priorities are not violated?

Pause the video!

Write your answer in a word file previously saved as <my_roll_no>_lecture_2.

When the interrupt occurs, process A transitions from blocked to ready and immediately preempts process B (running $\rightarrow$ ready), so A moves to running state.

Task# 4: Consider a background process that executes for x nanoseconds, then sleeps for y nanoseconds. It never terminates. Which transitions will it make according to the 5-state process model?

Pause the video!

Write your answer in a word file previously saved as `<my_roll_no>_lecture_2`.

The process will make the following transitions repeatedly:

→ Ready → Running → Blocked (Sleep) → Ready → Running → Blocked → ... (cycle continues forever)

Refining the 5-state process model

- If the system enforces priorities, the dispatcher can have multiple ready queues, one for each priority level.
- A new process is admitted to the end of the ready queue corresponding to its priority level.
- The dispatcher always picks the process at the head of the ready queue corresponding to the highest priority level.
- The dispatcher switches to a low priority ready queue if all higher priority ready queues are empty.

The need for swapping

- In a multiprogramming system, many processes are loaded in the main memory at the same time. If the currently running process makes an OS service call for I/O, it is blocked, and another ready process is chosen for execution. Consider a scenario where all processes in the main memory are blocked. What will the OS do now to prevent the processor from sitting idle?
- If we try to increase the size of main memory, it costs us money. Also, programmers tend to write larger programs with each passing day, so this is not an adequate solution.
- The OS chooses one or more processes and swaps them out to the hard disk. All contents of their address space are copied to the hard disk. Then their address spaces in main memory are deallocated. Now main memory has room for more processes. The OS creates new processes and starts to execute them.
- A process swapped out to the hard disk is called a **suspended** process. It can not execute as it is not loaded in main memory. The **suspended** queue contains processes swapped out to disk.
- If an I/O event occurs, the OS marks the process blocked for it as ready. Afterwards, the OS may or may not swap it into the main memory. So a suspended queue may contain both blocked and ready processes.
- Swapping involves I/O, i.e. writing to or reading from the hard disk. So it always takes time. The more we swap processes in and out of memory, the slower the system becomes. Since disk I/O is generally the fastest I/O in the system, swapping mostly enhances performance by reducing processor idle time.

Virtual Memory

- **The size of the code is limited by the size of the main memory.**
- **Virtual memory** is created by using part of the hard disk as an extension of the main memory.
- A process becomes executable when all its code is loaded in the virtual memory.
- Typically, all the process is loaded in the area of the hard disk that is mapped in virtual memory. The process is given a small address space in main memory. The process is divided into chunks called **pages**. The page currently being executed is loaded in the main memory. So the process is **partially** loaded in the main memory. When execution of this page completes, the OS swaps it out and swaps the next page into main memory. **We can now execute programs larger than the size of RAM in the system.**
- In a system using virtual memory, a **suspended** process is one which is only loaded on the hard disk and not even a single page is in the main memory.

Reasons for process suspension



SWAPPING



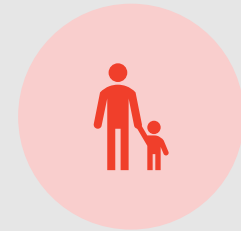
OTHER OS
REASON



INTERACTIVE USER
REQUEST



TIMING



PARENT PROCESS
REQUEST



Table 3.3 Reasons for Process Suspension

Swapping	The OS needs to release sufficient main memory to bring in a process that is ready to execute.
Other OS reason	The OS may suspend a background or utility process or a process that is suspected of causing a problem.
Interactive user request	A user may wish to suspend execution of a program for purposes of debugging or in connection with the use of a resource.
Timing	A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next time interval.
Parent process request	A parent process may wish to suspend execution of a descendent to examine or modify the suspended process, or to coordinate the activity of various descendants.



Reasons for
process
suspension

States in the 7-state process model



New



Ready



Blocked



Ready/Suspend



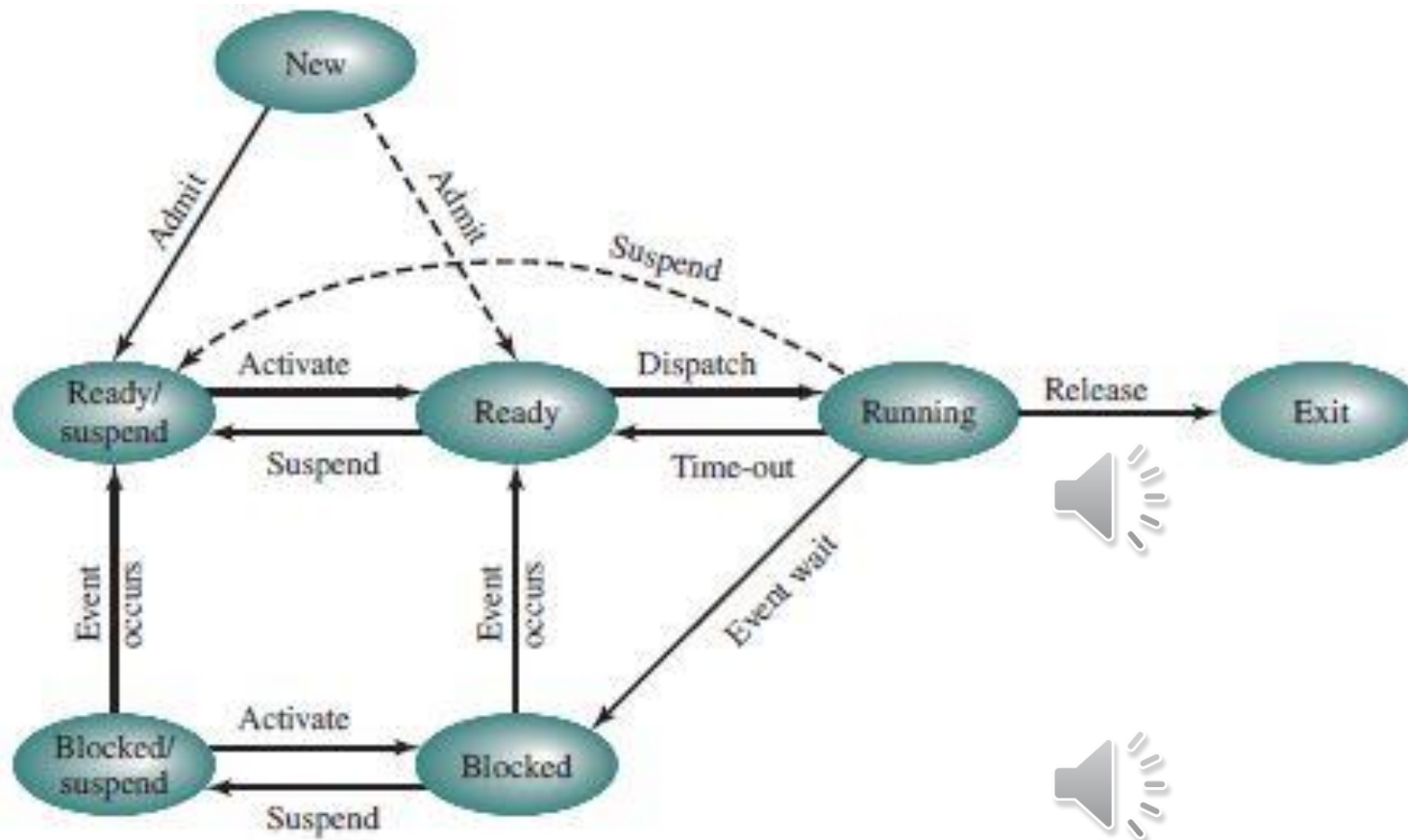
Blocked/Suspend



Running



Exit



(b) With two suspend states

Figure 3.9 Process State Transition Diagram with Suspend States

7-state process model

- We introduce two more queues.
- A process is in the ready/suspend queue when it is loaded on the hard disk, and not waiting for any event.
- A process is in the blocked/suspend queue when it is loaded on the hard disk and waiting for an event.

Concluding remarks

- In this lecture we introduced the concept of a process. A process is a program in execution.
- We saw the reasons due to which a new process is created, and the reasons which can lead to its termination.
- We introduced the concept of virtual memory, and suspension.
- We traced the life of a process from birth to death and explored the different states that it can have in an OS.

Things to do

- Read the book.
- I will ask you to submit the word file previously saved as <my_roll_no>_lecture_2 in  Google classroom.
- Write your questions on a piece of paper and keep it in front of you during the online session.
- If you have suggestions relevant to content or quality of this lecture, email them to me.